

Universidad Rafael Landívar.
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas.
Manejo e Implementación de Archivos
Ing. Manuel Rojas



PROYECTO 1

Santiago Jéfté Batz Rodríguez - 1548725

Quetzaltenango, 16 de septiembre de 2026

Descripción general

Se desarrolló una aplicación de escritorio en Python con PySide6 para administrar una configuración de usuario de forma persistente y segura, acá la interfaz permite modificar nombre de usuario, tema claro u oscuro, idioma, tamaño de fuente, color de la barra de menú, color de letra y fotografía de perfil, donde la parte central del proyecto es el manejo robusto de archivos, la configuración se carga al iniciar, se guarda con una estrategia temporal antes de reemplazar el archivo principal, se conserva un respaldo de la versión anterior y se controlan los casos de archivo ausente, archivo inválido y falta de permisos.

Funcionalidades implementadas

- Menú principal con Archivo, Edición y Ver como opciones simuladas, y Settings como opción funcional
- Configuración del nombre de usuario, tema, idioma, tamaño de fuente y colores
- Selección de fotografía de perfil desde el sistema de archivos y copia interna de la imagen
- Carga automática de config.json al iniciar la aplicación.
- Uso de valores predeterminados cuando el archivo principal no existe
- Guardado seguro mediante config.tmp y os.replace()
- Creación de config.bak con la última configuración válida anterior
- Recuperación desde config.bak cuando config.json está corrupto y el respaldo es válido
- Manejo controlado de errores de lectura y escritura
- Lectura y escritura explícita en UTF-8

Formato de almacenamiento: JSON

Se eligió JSON porque la configuración contiene distintos tipos de datos y cada valor queda identificado por su nombre, además, Python permite leerlo y escribirlo fácilmente.

Comparación entre JSON y CSV para la decisión

Aspecto	JSON	CSV
Lectura	Es fácil de leer porque cada dato tiene su nombre	Es fácil para tablas, pero no tanto para una configuración
Uso en Python	Se puede leer y guardar directamente con <code>json.load()</code> y <code>json.dump()</code>	Hay que trabajar con filas y columnas
Datos	Guarda texto y números de forma clara	Generalmente los datos se leen como texto
Organización	Se adapta bien a datos diferentes como nombre, tema, idioma y colores	Funciona mejor cuando todos los datos tienen la misma estructura
Tamaño	Puede ocupar un poco más de espacio	Normalmente ocupa menos espacio
Errores	Si el archivo está mal escrito, Python puede detectar que el JSON no es válido	Puede necesitar más revisiones para saber si los datos están correctos

CSV habría sido suficiente para datos estrictamente tabulares, pero JSON resulta más claro, extensible y apropiado para una configuración de usuario con propiedades de tipos diferentes.

Ejemplo de configuración generada

El archivo principal se almacena en data/config.json, y un ejemplo real utilizado en las pruebas fue:

```
{
  "nombre_usuario": "Joaquín Ordoñez",
  "tema_interfaz": "oscuro",
  "idioma": "es-ES",
  "tamaño_fuente": 14,
  "color_barra_menu": "#3F7469",
  "color_letra": "#F8FBF9",
  "foto_perfil": "data/profiles/profile_....png"
}
```

Los caracteres de “Joaquín Ordoñez” se conservan de forma legible, por lo que esta misma evidencia permite comprobar el uso correcto de UTF-8.

Escritura segura

La aplicación no sobrescribe config.json directamente, primero escribe la configuración nueva en config.tmp, fuerza el vaciado del contenido y, al final, utiliza os.replace() para sustituir el archivo principal. De esta forma se reduce el riesgo de dejar config.json incompleto si ocurre un cierre inesperado durante el guardado.

```
with open(self.temp_path, "w", encoding="utf-8") as archivo:
    json.dump(config.to_dict(), archivo, indent=4, ensure_ascii=False)
    archivo.flush()
    os.fsync(archivo.fileno())
```

```
os.replace(self.temp_path, self.config_path)
```

Y si la operación falla, el temporal se elimina de forma controlada y al iniciar la aplicación también se limpia un config.tmp sobrante de una ejecución interrumpida.

Funcionamiento de config.bak

Antes de reemplazar una configuración principal válida, la aplicación conserva una copia de la versión anterior como config.bak, por ello, el respaldo no contiene necesariamente los mismos valores que config.json, pues config.json representa el estado actual y config.bak conserva el estado válido inmediatamente anterior.

```
with open(self.config_path, "rb") as archivo_original:
    with open(self.backup_path, "wb") as archivo_backup:
        while True:
            bloque = archivo_original.read(8192)
            if not bloque:
                break
            archivo_backup.write(bloque)
```

Ejemplo: si config.json contiene Joaquín Ordoñez, y otros datos como 14 px, config.bak puede conservar Santiago Batz con un tamaño de fuente de 20 px, que era la configuración anterior.

Como protección adicional, antes de reemplazar el respaldo se valida que config.json sea legible y tenga un formato aceptado, si el archivo principal ya está corrupto, no se copia sobre un config.bak válido, de esta forma se evita perder el último respaldo útil.

Recuperación desde el respaldo

Si config.json existe pero no contiene JSON válido, se intenta leer config.bak, si el respaldo es válido, sus datos se cargan en memoria y se reconstruye config.json mediante el mismo mecanismo temporal, la interfaz informa “Configuración recuperada”

```
recovered = self._recover_from_backup()
```

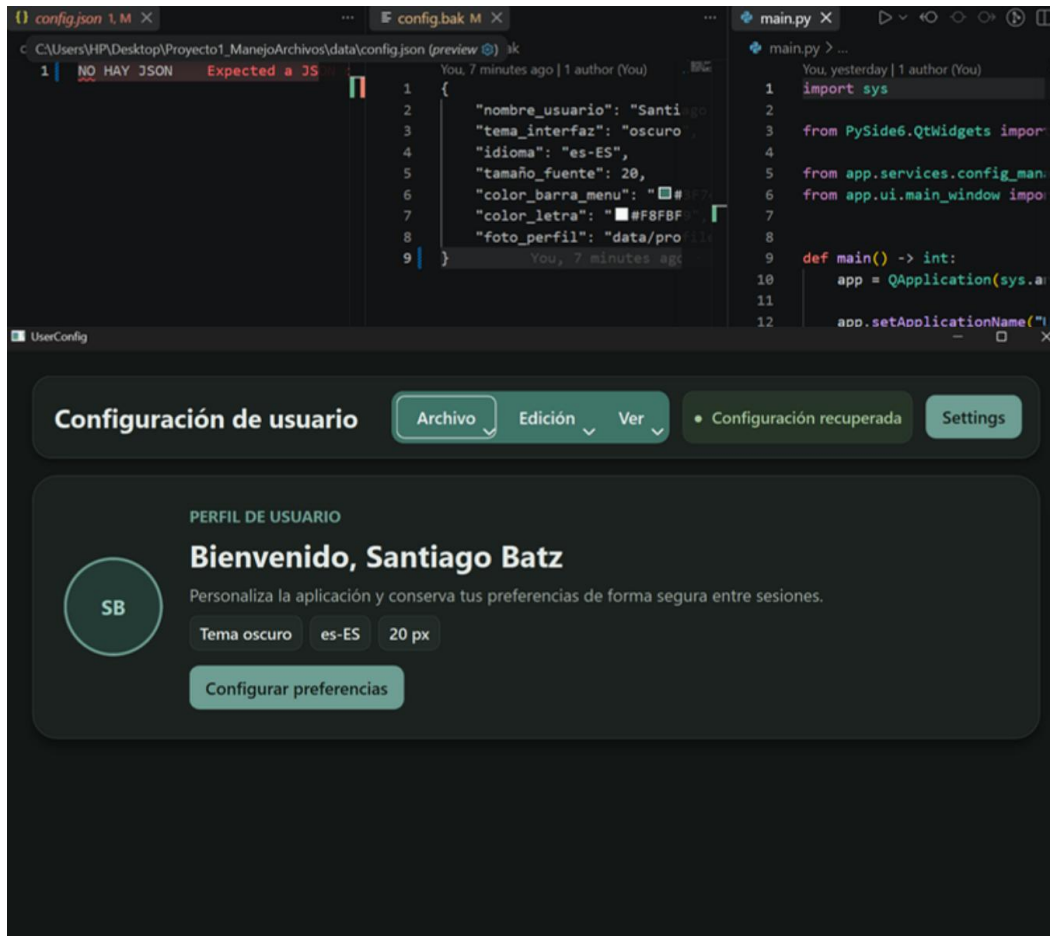
if recovered is not None:

```
    return recovered, "Configuración recuperada"
```

```
return UserConfig(), "Configuración inválida"
```

Así:

JSON corrupto + backup válido nos lleva a una configuración recuperada



JSON principal corrupto y respaldo válido, la aplicación recupera la configuración desde config.bak.

Manejo de errores y evidencias

Caso	¿Qué hace la aplicación?
El archivo no existe	La aplicación inicia con valores quemados
El archivo está dañado	Intenta recuperar los datos desde config.bak, y si no puede, utiliza los valores quemados
No hay permisos	Muestra un mensaje de error y evita que la aplicación se cierre inesperadamente

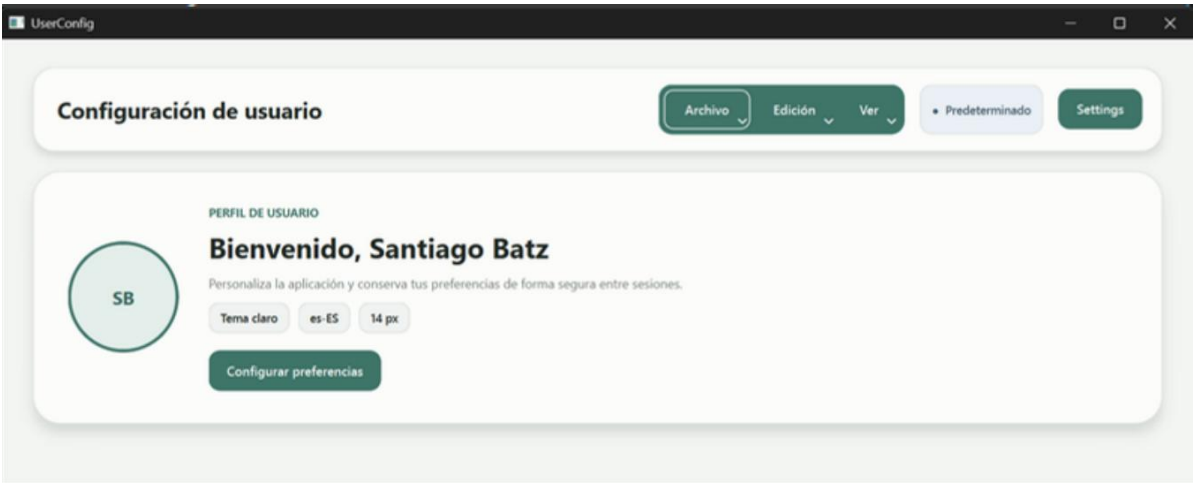
De esta forma, un problema con los archivos no provoca que la aplicación se cierre mostrando un error sin controlar

Archivo ausente: para esta prueba se retiraron temporalmente config.json y config.bak. Al iniciar, la aplicación no encuentra el archivo principal, utiliza los valores predeterminados de UserConfig y muestra el estado “Predeterminado”

Archivo ausente, nos manda a datos predeterminados o quemados



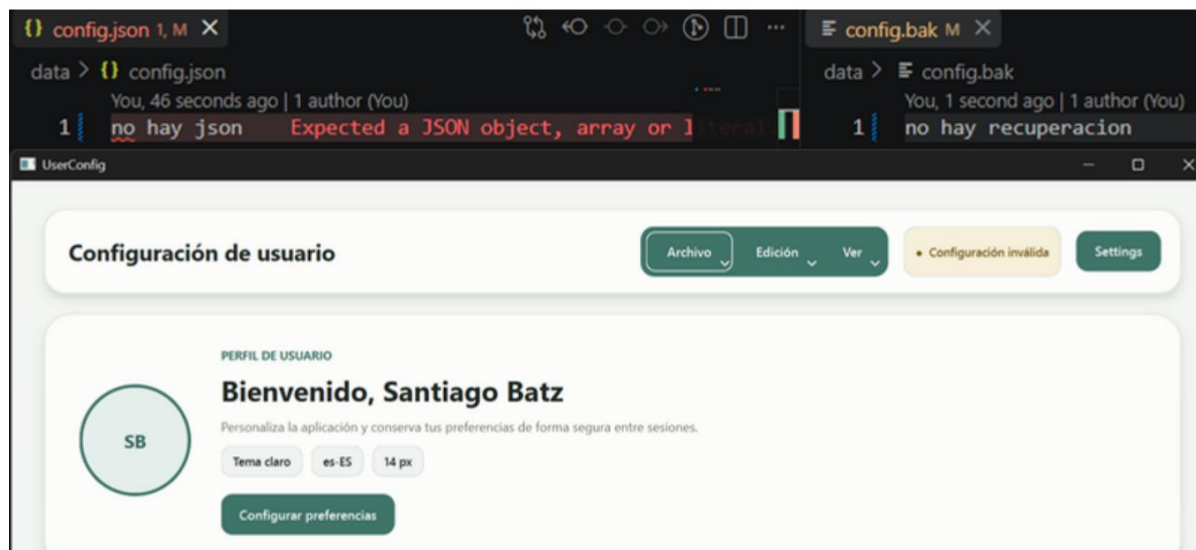
No hay nada en la carpeta



Entonces, con archivo ausente, el inicio correcto con valores predeterminados (el programa lo dice)

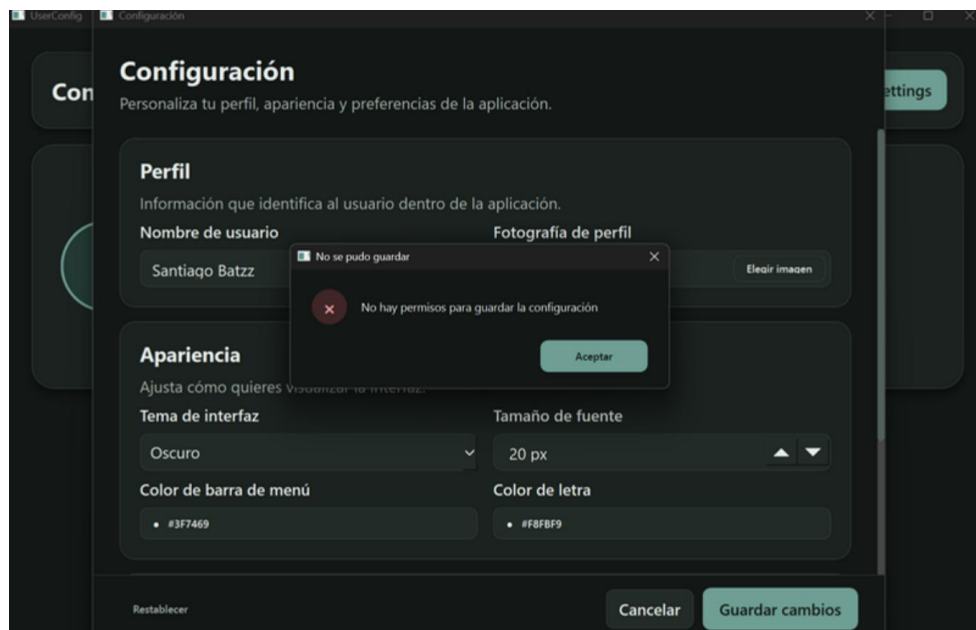
Archivo inválido sin respaldo válido: en esta prueba config.json contiene texto que no representa un objeto JSON válido y config.bak tampoco ofrece un respaldo recuperable. La aplicación detecta la corrupción, evita cerrarse y vuelve a los valores predeterminados, diferenciando el caso mediante el estado “Configuración inválida”.

JSON corrupto sin un backup válido, nos muestra una configuración inválida



Acá con el JSON inválido sin un backup válido, muestra los valores quemados, pero dice que la configuración es inválida

Falta de permisos: se provocó una situación de escritura sin permisos y posteriormente se intentó guardar desde Settings, la aplicación interceptó `PermissionError` y mostró un mensaje de error controlado en vez de tronarse el programa



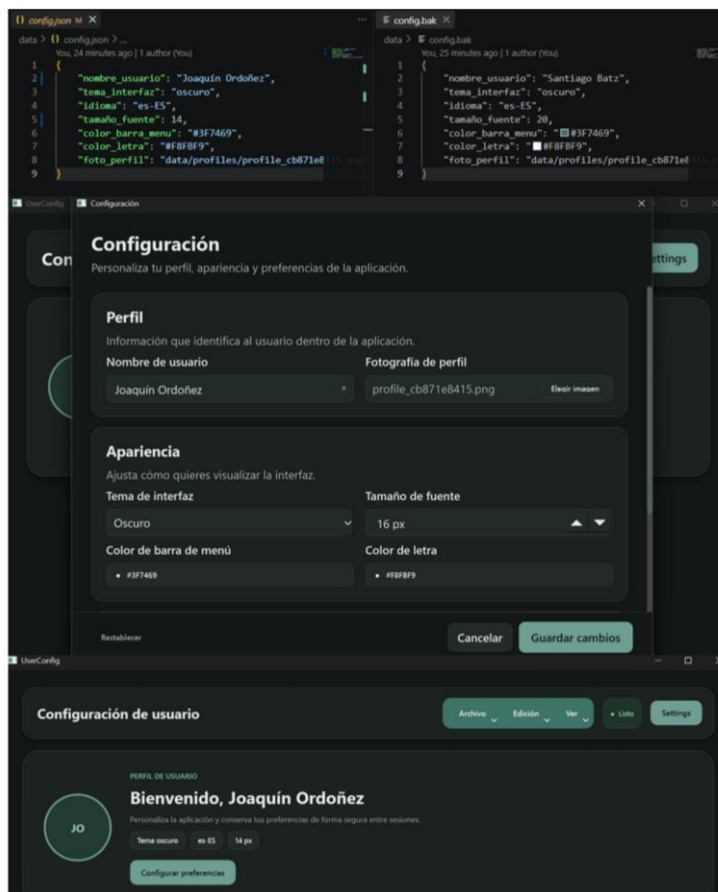
El manejo de falta de permisos al intentar guardar la configuración

Codificación UTF-8

Los accesos de texto a la configuración utilizan explícitamente `encoding="utf-8"`. En la prueba se utilizó el nombre “Joaquín Ordoñez”, en `config.json` puede observarse que la *í* acentuada y la *ñ* permanecen correctamente almacenadas y luego se muestran de nuevo en la aplicación.

```
with open(self.config_path, "r", encoding="utf-8") as archivo:  
data = json.load(archivo)
```

```
json.dump(  
config.to_dict(),  
archivo,  
indent=4,  
ensure_ascii=False,  
)
```



Se nota UTF-8, junto con la comparación entre la configuración actual y el respaldo anterior

Fotografía de perfil y archivos binarios

La fotografía elegida mediante el selector de archivos se copia al directorio data/profiles/. La copia utiliza los modos binarios rb y wb, por lo que se trasladan los bytes de la imagen sin convertirlos a texto. En config.json se guarda una ruta interna del proyecto y no la ubicación original del archivo seleccionado.

with open(source, "rb") as original:

with open(destination, "wb") as copy:

while True:

block = original.read(8192)

if not block:

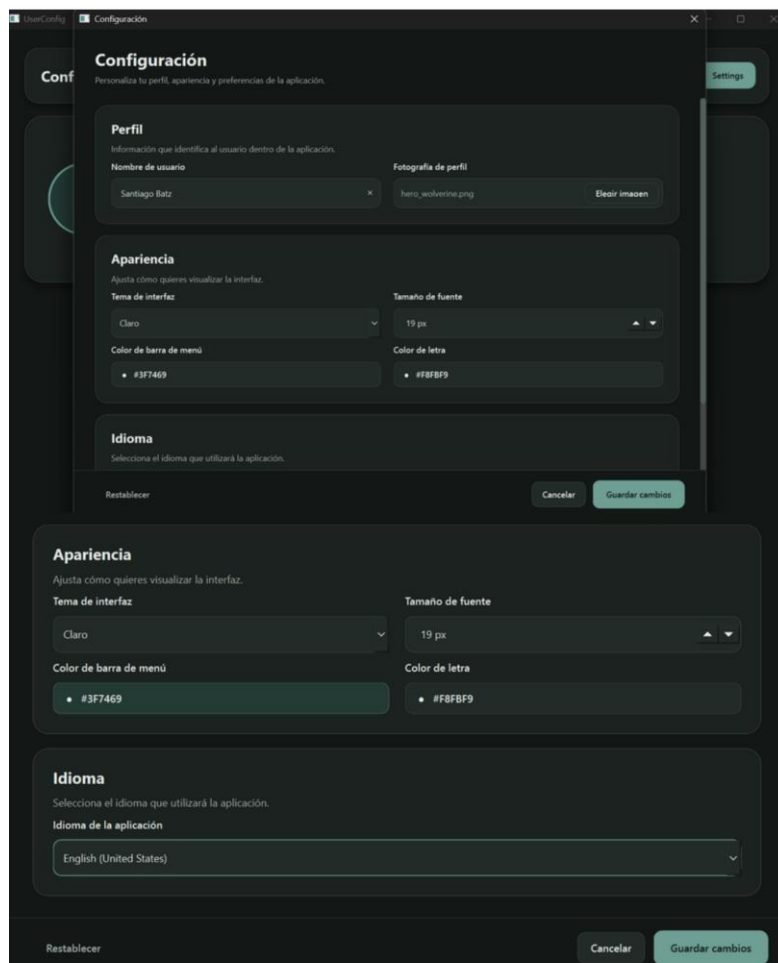
break

copy.write(block)

Y esto permite que la fotografía siga disponible, aunque la imagen original sea movida, si la ruta almacenada no puede resolverse, la interfaz utiliza las iniciales del nombre como alternativa

Funcionamiento de Settings

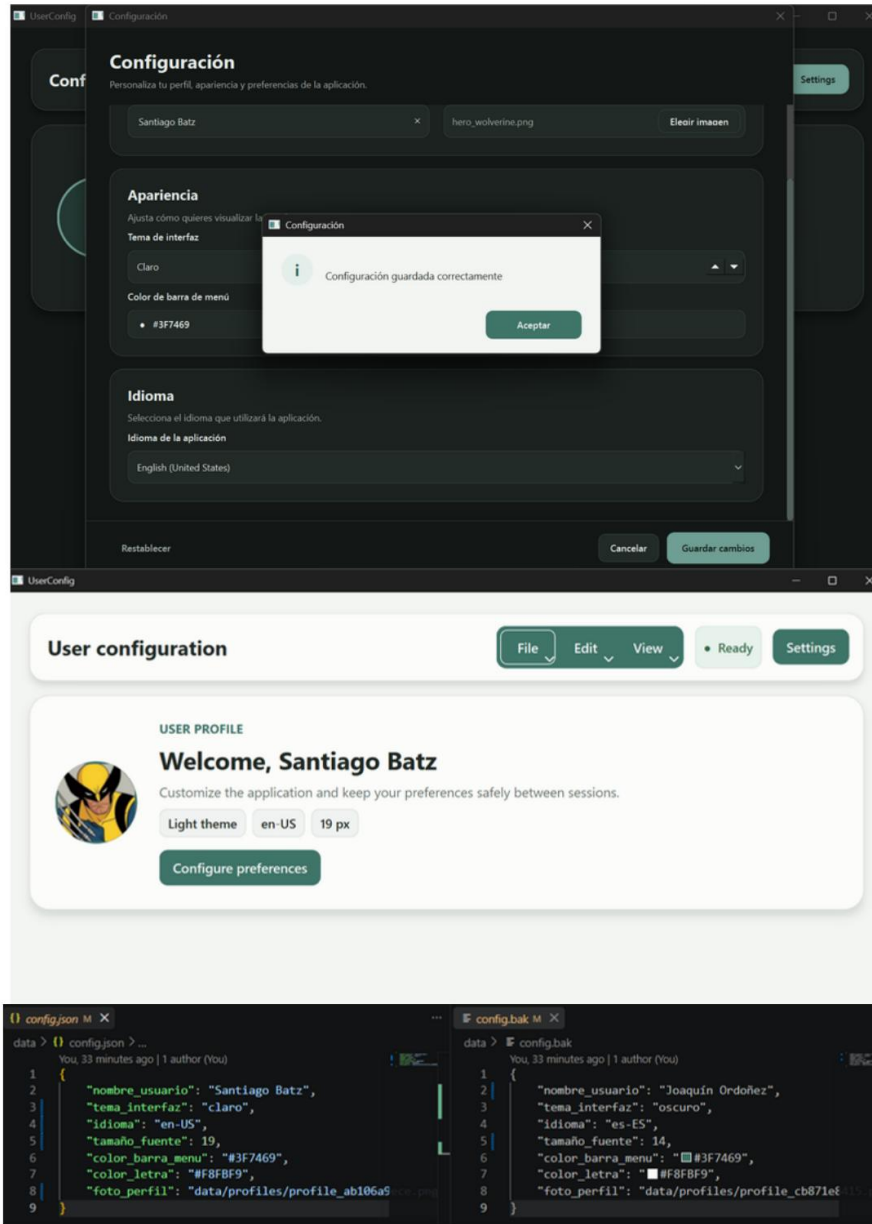
Settings concentra las operaciones reales de configuración, desde esta ventana se modifican nombre de usuario, fotografía, tema, tamaño de fuente, color de barra, color de letra e idioma, y al guardar, todos los valores recorren el mismo flujo de persistencia hacia config.tmp, config.bak y config.json.



Y la ventana Settings mira los controles de perfil, apariencia e idioma.

Guardado y aplicación inmediata de preferencias

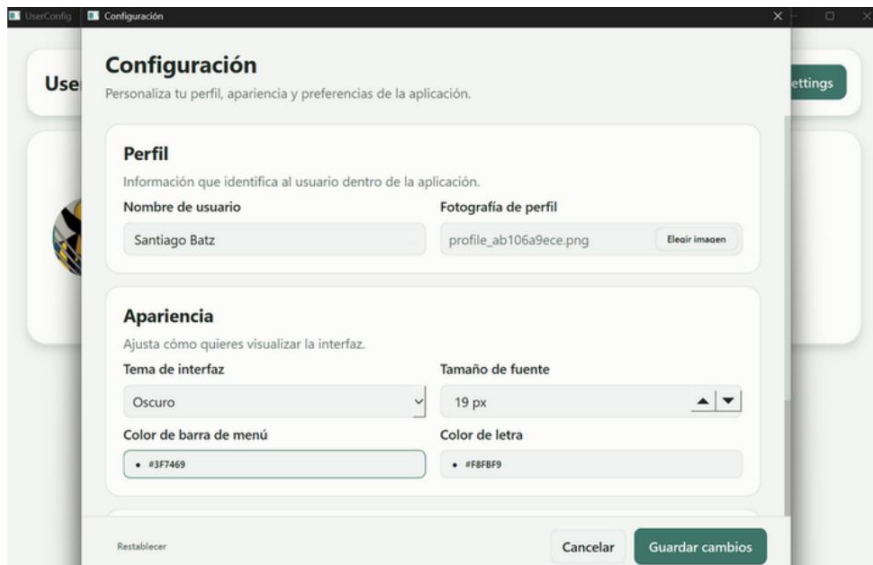
Después de guardar, la ventana principal recibe la nueva configuración y reconstruye su interfaz para reflejar tema, idioma, tamaño de fuente, colores y fotografía sin necesidad de reiniciar, la aplicación también conserva esos cambios para la siguiente ejecución.



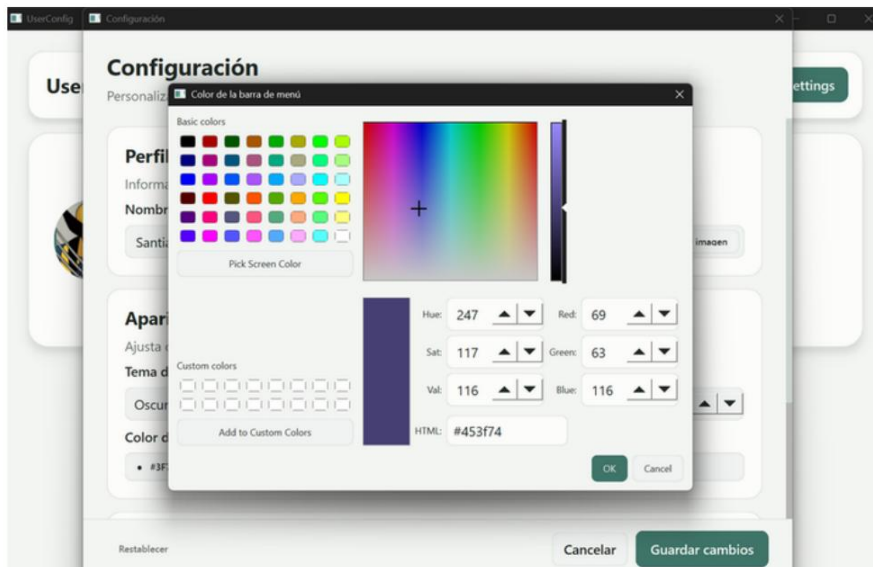
Selectores de color

Los colores de la barra de menú y del texto se seleccionan mediante QColorDialog, un selector propio de PySide6. Los valores elegidos se guardan en formato hexadecimal dentro del mismo archivo JSON y se aplican a los estilos de la interfaz.

Prueba de color de barras

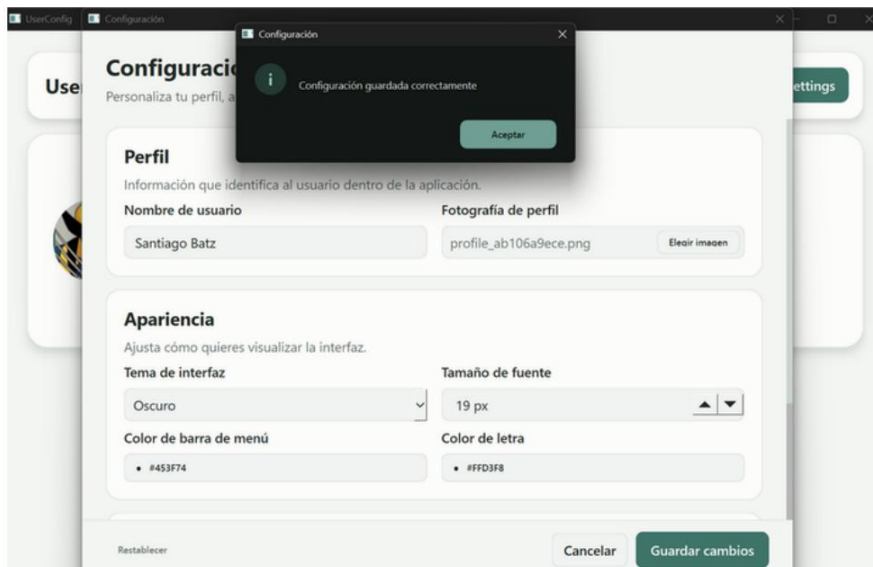
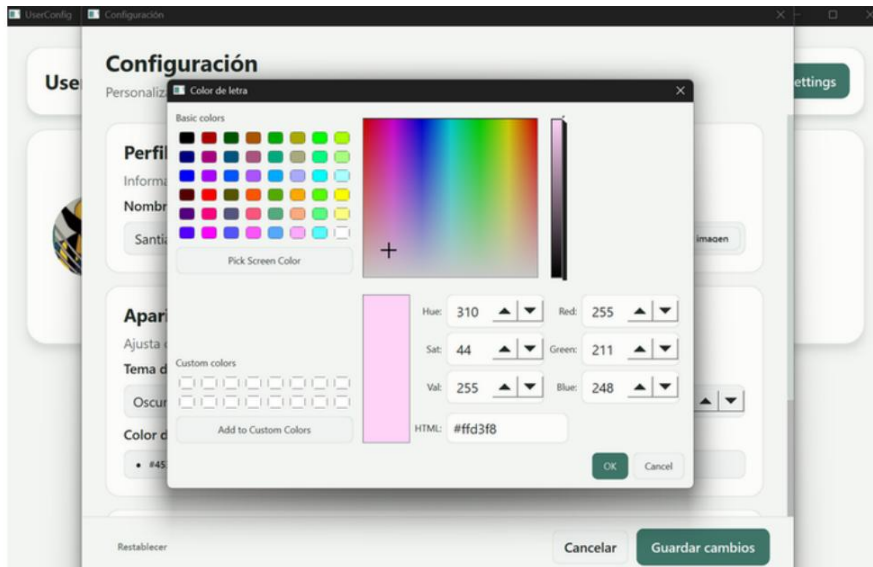


COLOR DE BARRA



Se observa la selección gráfica del color de la barra de menú

Color de letra

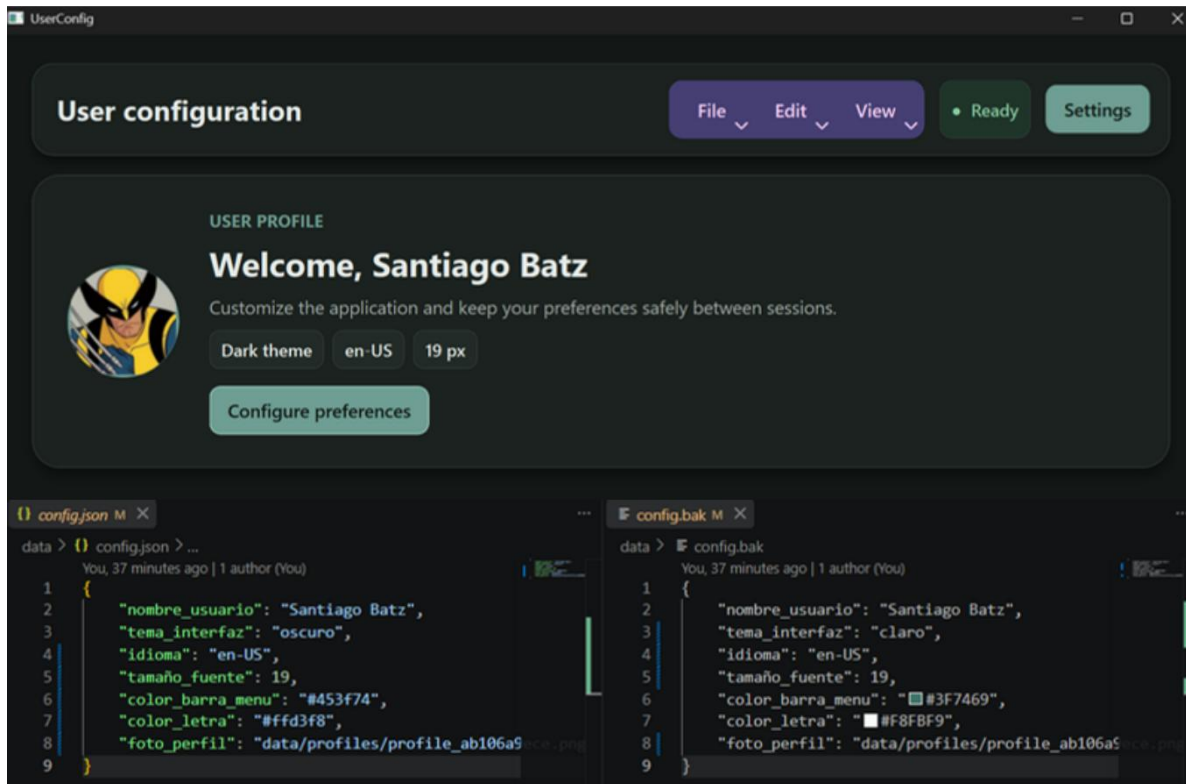


Se observa la selección gráfica del color de letra

Pantalla principal y menús

La pantalla principal resume la configuración activa y ofrece acceso a Settings. Archivo, Edición y Ver se implementan como menús simulados, tal como se solicita, y de igual forma las opciones responden de forma controlada, por lo que únicamente Settings y “Configurar preferencias” sí abren la ventana funcional que ejecuta las operaciones de persistencia.

CAMBIO DE COLORES REFLEJADO



Así se observa la interfaz principal con tema, idioma, fotografía y colores personalizados aplicados.

Conclusiones

El desarrollo de esta aplicación permitió trabajar de forma práctica con la lectura, escritura y actualización de archivos de configuración en Python, donde se usó JSON porque permite guardar los datos de forma ordenada, fácil de leer y sencilla de manejar desde el programa. También se implementó un sistema de respaldo con config.bak y una escritura segura con config.tmp, lo que ayuda a proteger la configuración anterior y evita que el archivo principal quede incompleto o dañado durante el guardado.

El proyecto puede responder de forma controlada cuando el archivo no existe, está dañado o no se tienen permisos para trabajar con él, además el uso de UTF-8 permitió guardar correctamente nombres con tildes y la letra ñ, por último, la ventana de configuración permite cambiar las preferencias del usuario y guardar esos cambios para que se mantengan al volver a abrir la aplicación.